CodeClash.AI

A PROJECT REPORT
for
Major Project-1 (CA301P)
Session (2025-26)

Submitted by

Mayank Srivastava
202410116100118
Mohammad Daud
202410116100121

Submitted in partial fulfilment of the
Requirements for the Degree of

MASTER OF COMPUTER APPLICATION

Under the Supervision of
Dr. Ankit Verma
Associate Professor

Submitted to

DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206

(DECEMBER 2025)

CERTIFICATE

Certified that **Mayank Srivastava 202410116100118, Mohammad Daud 202410116100121** have carried out the project work having "**CodeClash.AI**" (**CA301P**) for **Master of Computer Application** from KIET Group of Institutions (an Autonomous college) under Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Date:

Mayank Srivastava (202410116100118)
Mohammad Daud (202410116100121)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Dr. Ankit Verma
Associate Professor
Department of Computer Applications
KIET Group of Institutions, Ghaziabad

Dr. Sachin Malhotra
Dean (MCA)
Department of Computer Applications
KIET Group of Institutions, Ghaziabad

CodeClash.AI

Mayank Srivastava, Mohammad Daud

ABSTRACT

Learning programming effectively requires not only solving problems but also understanding the underlying concepts and logic behind solutions. Many existing coding platforms focus heavily on problem-solving and competition, often neglecting conceptual clarity and personalized guidance. This creates a gap between practice and true understanding, especially for beginners and intermediate learners.

CodeClash.AI is an **AI-powered coding and learning platform** designed to bridge this gap by integrating artificial intelligence directly into the coding environment. The platform enables users to practice coding problems using an in-browser code editor while receiving intelligent assistance through AI-driven editorials and a context-aware chatbot. By leveraging the **Gemini AI API**, CodeClash.AI provides step-by-step problem explanations, optimized solution approaches, and real-time doubt resolution tailored to the user's code and problem context.

The system is developed using a modern **MERN stack**, with **ReactJS** for the frontend, **Node.js and Express.js** for backend services, and **MongoDB Atlas** for secure and scalable data storage. Authentication is handled using **JWT-based security**, ensuring safe access to user-specific features. The integrated AI chatbot further enhances the learning experience by allowing users to ask natural language questions and receive immediate, relevant responses without leaving the coding interface.

CodeClash.AI transforms coding practice from a trial-and-error process into a guided, concept-driven learning experience. The platform promotes self-paced learning, improves problem-solving skills, and prepares students for technical interviews and academic assessments. This project demonstrates the practical application of artificial intelligence in education and highlights the potential of AI-assisted learning systems in modern software engineering.

ACKNOWLEDGEMENT

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor, **Dr. Ankit Verma** for his guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions.

Words are not enough to express my gratitude to **Dr. Sachin Malhotra, Professor and Dean**, Department of Computer Applications, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot on many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kind of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Mayank Srivastava

Mohammad Daud

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

INTRODUCTION

1.1 Background of the Problem

In recent years, the demand for software development and programming skills has increased significantly due to rapid advancements in technology and digital transformation across industries. Coding proficiency has become an essential skill for students and professionals pursuing careers in computer science, information technology, and related fields. However, many learners struggle to build strong problem-solving and algorithmic thinking abilities despite having access to numerous online resources.

Traditional methods of learning programming, such as classroom-based teaching and textbook-oriented approaches, often focus on theoretical concepts with limited hands-on practice. While these methods provide foundational knowledge, they fail to offer continuous, personalized guidance during practical problem-solving.

To address this gap, several online coding platforms have emerged that allow users to practice programming problems. Although these platforms help learners improve their coding speed and exposure to problem sets, they largely emphasize problem completion and competition rather than conceptual understanding. Learners are often left without clear explanations for incorrect solutions, forcing them to rely on external tutorials, forums, or trial-and-error methods.

With recent advancements in **Artificial Intelligence (AI)** and **Large Language Models (LLMs)**, intelligent systems are now capable of understanding programming problems, analyzing code logic, and generating human-like explanations. These technologies have demonstrated strong potential in educational applications, including automated tutoring, code analysis, and interactive learning support.

Despite this progress, the integration of AI directly into coding practice environments remains limited. There is a pressing need for an intelligent, interactive, and accessible platform that combines **hands-on coding practice** with **AI-driven guidance and real-time conceptual support**. **CodeClash.AI** aims to address this need by embedding AI-powered learning assistance directly within the coding workflow, making programming education more effective, engaging, and learner-centric.

1.2 Problem Statement

1.2.1 Limited Access to Personalized Coding Guidance

Many learners face difficulties in accessing quality programming guidance due to limited availability of mentors and instructors. Students from non-technical backgrounds, rural areas, or self-learning environments often lack immediate support when they encounter logical or conceptual challenges while solving coding problems. This absence of personalized guidance slows learning and reduces confidence.

1.2.2 Limitations of Existing Coding Platforms

Most existing online coding platforms primarily focus on problem-solving and competition. While they provide large problem repositories, they offer limited support for conceptual understanding. Explanations are often static, generic, or completely absent, forcing learners to search external resources such as blogs or videos, which disrupts the learning flow.

1.2.3 Difficulty in Debugging and Conceptual Errors

Learners frequently struggle to identify logical mistakes, edge cases, or inefficient approaches in their code. Without real-time feedback or explanatory insights, students resort to trial-and-error methods. This leads to shallow learning, where problems may be solved without fully understanding the underlying logic or algorithm.

1.2.4 Lack of Intelligent and Context-Aware Learning Systems

Despite advances in artificial intelligence, most coding platforms do not effectively integrate AI to assist learners in real time. Existing tools fail to analyze the problem context and user-written code together to provide meaningful, context-aware explanations, debugging help, or solution guidance.

1.2.5 Need for an AI-Driven and Accessible Learning Platform

There is a strong need for an intelligent, accessible, and learner-centric platform that seamlessly combines coding practice with AI-based guidance. Such a system should provide problem editorials, step-by-step explanations, and interactive doubt resolution directly within the coding environment. **CodeClash.AI** aims to fulfil this requirement by leveraging AI to deliver personalized, real-time learning assistance, making programming education more effective and accessible for all learners.

1.3 Aims and Objectives

1.3.1 Aim of the Project

The primary aim of this project is to **design and develop an AI-powered coding and learning platform** that enables students to practice programming problems while receiving intelligent, real-time conceptual guidance. The platform focuses on improving problem-solving skills, algorithmic thinking, and code understanding by integrating artificial intelligence directly into the coding environment.

1.3.2 Objectives of the Project:

The objective of the **CodeClash.AI** project is to design and develop an intelligent, AI-powered coding and learning platform that assists students in understanding programming concepts while practicing coding problems in a structured environment. The platform aims to integrate artificial intelligence, specifically the **Gemini AI**, to provide problem editorials, step-by-step solution approaches, and optimized code implementations directly within the coding interface. In addition, the system incorporates a context-aware AI chatbot that enables users to ask coding-related queries and receive instant, relevant responses based on the problem and their submitted code.

By combining hands-on coding practice with real-time AI-driven guidance, the platform reduces reliance on external learning resources and promotes deeper conceptual clarity. The system is designed using modern web technologies to ensure security, scalability, and ease of use, while also enabling users to track their progress and learning history. Overall, CodeClash.AI aims to make programming education more accessible, effective, and learner-centric by transforming traditional problem-solving into a guided and interactive learning experience.

Key Objectives

1. **AI-Driven Coding Guidance and Learning Support**
    - Develop an AI-powered system that provides editorials, step-by-step explanations, and optimized solutions for coding problems.
    - Assist users in progressing from basic to advanced problem-solving through structured AI guidance.

2. **Real-Time Pose Detection and Movement Analysis**
    - • Analyze user-written code along with problem context to identify logical, syntactical, and conceptual issues.
    - • Provide intelligent assistance based on execution behavior and algorithmic approach
3. **Personalized Feedback and Skill Improvement**
    - Deliver instant, AI-generated feedback on code correctness, efficiency, and logical structure.
    - • Suggest improvements and alternative approaches to strengthen conceptual understanding
4. **User Progress Tracking and Performance Analytics**
    - Track coding activity, submissions, and AI interactions across multiple learning sessions.
    - • Maintain learning history to help users monitor progress and identify improvement areas.
5. **Accessibility and Multi-User Support**
    - Ensure the platform is accessible through standard web browsers without requiring specialized hardware or software.
    - • Design the system to support learners from different academic backgrounds, skill levels, and geographical locations.

1.4 Scope of the Projecct

The development of **CodeClash.AI**, an AI-powered coding and learning platform, is highly relevant in today’s technology-driven world where programming skills are essential for academic success and career growth. As the demand for software developers continues to rise, many students struggle to gain strong problem-solving and algorithmic understanding due to the lack of personalized guidance and effective learning tools. Traditional classroom teaching and existing coding platforms often focus on theoretical concepts or competitive problem-solving, which limits deep conceptual learning. This project addresses these challenges by leveraging **Artificial Intelligence (AI)** to create an intelligent, interactive, and accessible coding learning environment that supports students throughout their learning journey.

Areas of Importance:

1. **Improved Conceptual Understanding**
   The platform provides AI-driven editorials, step-by-step explanations, and optimized solutions that help learners understand the logic and approach behind coding problems. This enhances conceptual clarity and reduces reliance on rote memorization or trial-and-error methods

2. **Enhanced Accessibility and Inclusivity**
   As a web-based platform, CodeClash.AI can be accessed anytime and from anywhere with an internet connection. This makes quality coding education available to students from urban and rural areas alike, regardless of their access to mentors or coaching institutes.

3. **Intelligent and Context-Aware Assistance**
   By integrating **Gemini AI**, the platform analyzes both the problem statement and user-written code to provide relevant, context-aware guidance. This allows learners to receive instant help, debug errors, and clarify doubts directly within the coding environment.

4. **Continuous Learning and Performance Tracking**
   The system records user submissions, AI interactions, and learning history across multiple sessions. This enables users to monitor their progress over time, identify weak areas, and improve their coding skills systematically.

5. **Support for Multiple Skill Levels**
   CodeClash.AI is designed to support learners at different stages, from beginners learning basic logic to advanced users practicing complex algorithms. The AI-driven guidance adapts to the user's level, ensuring that learning remains relevant and effective.

6. **Technological Innovation in Programming Education**
   This project demonstrates how modern AI technologies can enhance traditional programming education. By embedding AI directly into the coding workflow, CodeClash.AI showcases the practical application of intelligent systems to create interactive, personalized, and learner-centric educational experiences.

By providing a smart alternative to conventional coding practice methods, **CodeClash.AI bridges the gap between problem-solving and conceptual understanding**, empowering learners to become confident, independent, and skilled programmers.

1.5 Hardware and Software requirement

For the successful design and implementation of **CodeClash.AI**, both hardware and software components play an important role in ensuring smooth operation and an effective learning experience. The platform is designed with **minimal yet efficient requirements** so that it remains accessible to a wide range of users. CodeClash.AI can run seamlessly on commonly available devices such as laptops, desktops, and tablets, making it suitable for both academic and self-learning environments. The requirements ensure proper functioning of the web-based coding editor, AI-powered learning features, and secure data handling.

1.5.1 Hardware Requirements

The hardware specifications are selected to support smooth web interaction, code execution, and AI-assisted features without requiring high-end systems:

• **Laptop/Desktop/Tablet:** A standard computing device capable of running modern web browsers.

• **Processor:** Minimum Intel i3 / AMD Ryzen 3 (or equivalent) or higher for efficient browser performance and multitasking.

• **RAM:** At least 8 GB RAM to ensure smooth execution of the code editor and AI interactions.

• **Storage:** Minimum 128 GB HDD/SSD for operating system, browser cache, and development tools.

• **Internet Connection:** Stable broadband or 4G/5G internet connectivity to enable seamless communication with backend services and the Gemini AI API.

1.5.2 Software Requirements

- The software tools and technologies used in CodeClash.AI ensure efficient development, deployment, and execution of the platform:
- **Programming Languages:** JavaScript (ES6+) for frontend and backend development.
- **Frontend Technologies:** ReactJS, HTML5, CSS3, and Tailwind CSS for building responsive and interactive user interfaces.
- **Code Editor:** Monaco Editor for in-browser coding with syntax highlighting and auto-completion.
- **Backend Framework:** Node.js with Express.js for handling server-side logic and RESTful APIs.
- **Database:** MongoDB Atlas for secure, scalable, cloud-based data storage.
- **AI Integration:** Gemini AI API for AI Learn editorials and chatbot assistance.
- **Authentication:** JWT (JSON Web Tokens) for secure user authentication and session management.
- **Development Tools / IDE:** Visual Studio Code for development, debugging, and version control.
- **Testing Tools:** Postman and browser developer tools for API and functional testing.

These hardware and software requirements ensure **reliable performance, smooth AI-assisted learning, and a user-friendly coding experience**, making CodeClash.AI suitable for real-world educational use.

INTRODUCTION TO EXISTING SYSTEM

Existing Solutions

1. Traditional Classroom-Based Programming Education

- Programming concepts are taught through lectures, textbooks, and lab sessions in colleges and training institutes.
- Provides theoretical knowledge and limited hands-on coding practice under instructor supervision**.**

2. Online Coding Practice Platforms

- Platforms such as **LeetCode, CodeChef, and HackerRank** allow users to practice coding problems.
- Useful for improving problem-solving speed and exposure to different question types**.**

3. Online Tutorials and Video-Based Learning

- YouTube channels and e-learning websites provide recorded tutorials on programming concepts.
- Flexible learning options that allow students to learn at their own pace**.**

4. Coding Bootcamps and Short-Term Training Programs

- Institutions and private organizations conduct intensive coding bootcamps and workshops.
- Focused on fast-paced learning and interview preparation**.**

Limitations of Existing Solutions

1. **Limited Personalized Guidance**

    - Classroom teaching and online platforms often lack individual attention for each learner.

- • Students struggle to resolve doubts when instructors or mentors are unavailable.

2. **Fragmented Learning Exprience**

    - Learners frequently switch between coding platforms, videos, blogs, and discussion forums.
    - This breaks concentration and disrupts the learning flow.

3. **Lack of Real-Time Feedback**

    - Most coding platforms do not explain *why* a solution is incorrect or inefficient.
    - Learners receive verdicts like "Wrong Answer" without meaningful explanation

4. **One-Way Learning Approach**

    - Video tutorials and static content do not allow interactive doubt resolution.
    - Learners cannot ask contextual questions related to their own code.

5. **Time and Schedule Constraints**

    - Fixed class timings make it difficult for working individuals and students to attend regularly.

6. **Inconsistent Quality of Learning Resources**

    - Quality of tutorials and explanations varies widely across platforms.
    - Learners often receive incomplete or overly complex explanations.

7. **Lack of Learning Progess Tracking**

    - Many platforms focus on problem completion rather than learning improvement.
    - Users are unable to track conceptual growth, mistakes, or learning patterns over time.

2.2 Motivation

The motivation behind developing **CodeClash.AI**, an AI-powered coding and learning platform, arises from the growing demand for strong programming and problem-solving skills in today's technology-driven world. Many students and self-learners struggle to understand coding concepts despite practicing numerous problems, mainly due to the lack of personalized guidance and real-time support. Limited access to mentors, high coaching costs, and fragmented learning resources create a gap between the need to master coding skills and the ability to learn them effectively.

- **Social Motivation**
    - Increasing demand for programming skills among students and job seekers.
    - Need to make quality **coding education accessible to learners from diverse backgrounds.**
- **Technological Motivation**
    - Rapid advancement in Artificial Intelligence and Large Language Models.
    - Opportunity to use AI for contextual code analysis, explanations, and intelligent guidance.
- **Educational Motivation**
    - Lack of interactive and concept-driven coding learning platforms.
    - Need for AI-assisted systems that focus on understanding rather than rote problem-solving**.**
- **Personal Motivation**
    - Desire to help learners gain confidence in coding and problem-solving.
    - Aim to simplify the learning process by integrating guidance directly into the coding environment.
- **Professional Motivation**
    - Interest in applying AI technologies to real-world educational challenges.

- Opportunity to build an innovative full-stack project combining software development and artificial intelligence.

PROPOSED SYSTEM

3.1 Proposed Solution

The proposed solution is the development of **CodeClash.AI**, an AI-powered coding and learning platform that provides an intelligent, interactive, and accessible environment for students to practice and understand programming concepts. The system is designed to overcome the limitations of traditional coding learning methods by integrating **Artificial Intelligence (AI)** directly into a web-based coding platform. It enables users to solve coding problems, understand underlying concepts, and receive real-time guidance within a single, unified interface.

The platform works by allowing users to select coding problems and write solutions using an in-browser code editor. The system analyzes the user's code and problem context through AI-powered services. By leveraging **Gemini AI**, the platform evaluates logic, approach, and structure, and compares them with optimized reference solutions to identify mistakes, inefficiencies, and conceptual gaps.

Based on this analysis, CodeClash.AI provides **AI Learn editorials**, which include step-by-step explanations, algorithmic approaches, and optimized code implementations. In addition, a **context-aware AI chatbot** assists users by answering coding-related queries, debugging doubts, and explaining concepts in real time. Users can interact with the AI in natural language, making the learning experience more intuitive and effective.

The platform supports learners at different skill levels, ranging from beginners to advanced programmers. Users can practice problems at their own pace and receive instant feedback to refine their solutions. Performance tracking features store user submissions, AI interactions, and learning history, enabling users to monitor progress and improve continuously. This proposed system ensures that coding education is **affordable, flexible, personalized, and accessible**, helping learners build strong problem-solving skills and conceptual understanding

3.2 Key Features

Core Features

- AI-powered coding and learning platform integrated within a web-based environment.
- In-browser code editor for writing, running, and testing solutions.
- Problem selection based on topic and difficulty levels

Learning and Practice Features

- AI Learn feature providing editorials, step-by-step explanations, and optimized solutions.
- Support for beginners, intermediate, and advanced learners.
- Manual code execution and testing for better understanding of logic and output

AI Assistance and Analysis

- Context-aware AI chatbot for real-time doubt resolution.
- AI-based code analysis to identify logical and conceptual errors.
- Intelligent suggestions for improving code efficiency and approach.

User Management

- Secure user registration and login using JWT authentication.
- Personalized user dashboard displaying profile details and learning progress.
- Protection of user data and secure session management.

Technical Features

- Fully web-based platform accessible on laptops, desktops, and tablets.
- No additional software or specialized hardware required.
- Scalable system architecture using modern web technologies and cloud services.

3.3 Advantages Over Existing System

Traditional coding learning systems often face challenges such as lack of personalized guidance, fragmented learning resources, and limited conceptual explanations. Most existing platforms focus primarily on problem-solving or competition, offering static verdicts like “correct” or “wrong” without meaningful feedback. Learners are frequently required to rely on external tutorials, forums, or trial-and-error methods, which reduces learning efficiency and conceptual clarity. These limitations make it difficult for students to develop strong problem-solving skills and confidence in programming.

The proposed **CodeClash.AI** system overcomes these challenges by introducing an intelligent, interactive, and AI-driven learning environment. By integrating **Artificial Intelligence (Gemini AI)** directly into the coding workflow, the platform analyzes user code and problem context to provide real-time explanations, editorials, and personalized guidance. This ensures that learners not only solve problems but also understand the logic and approach behind each solution. The system makes coding education more effective, accessible, and learner-centric compared to traditional methods.

Key Advantages

Accessibility and Convenience

- Can be accessed from any location with an internet connection.
- No need for physical classrooms, coaching institutes, or fixed schedules

Cost Effectiveness

- Eliminates the need for expensive coaching programs and subscriptions.
- Uses widely available web technologies and AI-based automation

Advanced Technology Integration

- Real-time AI-based pose detection and movement analysis.
- Instant feedback and correction mechanism.

Improved Learning Experience

- Interactive and personalized training sessions.
- Allows self-paced learning with continuous guidance.

Performance Monitoring

- Tracks user progress and training history.
- Provides accuracy scores and performance analytics.

Scalability and Future Growth

- Can support a large number of users simultaneously.
- Easily expandable with new techniques and advanced AI features.

SYSTEM ARCHITECTURE

4.1 Overall System Design

The **CodeClash.AI** platform is implemented using a web-based, AI-centric client–server architecture that enables seamless coding practice, AI-driven analysis, and intelligent learning assistance. The system is designed to handle user interactions, code execution, and AI-based guidance efficiently while maintaining high responsiveness and scalability.

The architecture is divided into the following implementation layers:

1. Frontend Layer (Client Side)

- Developed as a responsive web application using modern frontend technologies.
- Provides an in-browser code editor for writing, running, and testing code.
- Allows users to browse coding problems and interact with AI features.
- Sends user code, problem context, and AI queries to the backend through REST API calls.
- Displays execution output, AI editorials, chatbot responses, and learning progress.

2. Backend & AI Processing Layer

- Acts as the core processing unit of the system.
- Handles:
  - User authentication and authorization
  - Problem management and retrieval
  - Code execution and submission handling
  - AI Learn and AI Chatbot request processing
- Integrates Gemini AI for contextual code analysis, editorial generation, and intelligent feedback.

3. Data Layer (Persistent Storage)

- Stores structured data including:

    - User credentials and profiles
    - Coding Problems and test cases
    - Code submissions and execution results
    - AI Learning Sessions and Chatbot History
- Enables progress tracking, learning analytics, and data retrieval.

This layered implementation ensures **scalability, modularity, and maintainability**, allowing AI components to be upgraded independently of the UI.

4.2 Architecture Diagram (Technical Description)

The architecture diagram illustrates the data flow during a coding and learning session:

1. The User accesses the platform through a web browser.
2. The Frontend Application allows the user to select a problem and write code using the in-browser editor.
3. User code and problem context are sent to the Backend Server via REST APIs.
4. The Code Execution Module processes the code and generates output or error results.
5. The AI Processing Module forwards relevant context to the **Gemini AI API** for editorial generation or chatbot responses.
6. The AI Engine analyzes the problem and user code and returns explanations, solutions, or answers.
7. The Database stores submissions, AI responses, and learning history.
8. The processed output, AI editorials, and chatbot feedback are returned to the frontend and displayed to the user.

The architecture supports asynchronous **request handling, secure communication**, and efficient **AI integration**, ensuring a smooth and intelligent coding and learning experience.

.

4.3 Modules Description

1. **Authentication & User Management Module**

    • Implements secure user registration and login using JWT-based authentication.

    • Controls access to protected features through token validation.

    • Maintains user profiles, preferences, and session information.

2. **Problem Management Module**

    • Manages the retrieval and display of coding problems.

    • Handles problem categorization based on topic and difficulty level.

    • Supplies problem statements, constraints, and test cases to other modules.

3. **Code Editor & Execution Module**

    • Provides an in-browser code editor for writing and testing solutions.

    • Executes user-submitted code and generates output or error responses.

    • Coordinates code execution results with submission storage and display.

4. **AI Learn Module**

    • Integrates with Gemini AI to generate editorials and solution explanations.

    • Provides step-by-step approaches and optimized code implementations.

    • Delivers concept-focused guidance to enhance problem understanding.

5. **AI Chatbot Module**

    • Enables real-time, conversational interaction between the user and AI.

    • Processes user queries related to code logic, errors, and concepts.

    • Generates context-aware responses using problem and code information.

6. **Submission & Result Management Module**

    • Stores user code submissions and execution outcomes.

    • Evaluates solution status such as correctness and completion.

    • Maintains submission history for learning review.

7. **Performance Tracking & Analytics Module**

    • Tracks user activity across multiple coding sessions.

    • Generates progress summaries and learning trends.

    • Helps users identify weak areas and improvement opportunities.

8. **Database Module**

    • Implements structured and scalable data storage using MongoDB.

    • Stores user data, problems, submissions, AI interactions, and results.

    • Supports efficient data retrieval, consistency, and persistence.

TECHNOLOGY STACK

5.1 Programming Languages: Javascript

JavaScript was chosen as the primary programming language for the development of **CodeClash.AI** due to its versatility, widespread adoption, and strong support for full-stack web development. JavaScript enables seamless development of both frontend and backend components using a single language, which simplifies system integration and maintenance. Its event-driven and asynchronous nature makes it well suited for building responsive web applications and handling real-time user interactions.

JavaScript plays a crucial role in implementing the interactive coding environment, managing user inputs, **handling API communication, and integrating AI services**. The language also enables smooth integration with modern frameworks and libraries used for building scalable and high-performance applications. By using JavaScript across the stack, CodeClash.AI **ensures consistency, faster development, and easier debugging**.

Key Advantages of Using Javascript:

- Single language for frontend and backend development
- Asynchronous and non-blocking execution model
- Strong ecosystem of frameworks and libraries
- Excellent support for web-based interactive applications
- Cross-platform compatibility
- Large developer community and continuous ecosystem growth

5.2 Framework and Libraries

1. **ReactJS**
    - ReactJS is a popular frontend library used for building dynamic and responsive user interfaces.

- In CodeClash.AI, ReactJS is used to design the user interface, manage component-based architecture, and handle state efficiently.

Key reasons for choosing ReactJS:

- Component-driven development for better code reusability
- Efficient state management using React Hooks
- Fast rendering through Virtual DOM
- Strong ecosystem and community support
- Seamless integration with modern UI libraries

2. Monaco Editor

- Monaco Editor is the code editor that powers Visual Studio Code.
- It is used in CodeClash.AI to provide an in-browser coding experience similar to professional development environments

  Its utilities enabled:
  - Syntax highlighting and code formatting
  - Auto-completion and error detection
  - Support for multiple programming languages
  - Enhanced coding experience for learners

3. Node.js

- Node.js is used as the backend runtime environment for CodeClash.AI.
- It enables efficient handling of client requests and API communication

  It helped in:
  - Event-driven and non-blocking I/O model
  - High performance for concurrent users
  - Seamless integration with JavaScript-based frontend
  - Scalable backend architecture

4. Express.js

- Express.js is a lightweight web framework built on Node.js.
- It is used to develop RESTful APIs for authentication, problem management, submissions, and AI interactions.

Key features of Express.js:

- Simplified routing and middleware support
- Secure request handling
- Easy integration with authentication mechanisms

5. Gemini AI API

• Gemini AI is integrated to power the AI Learn and AI Chatbot features of CodeClash.AI.

• It provides intelligent editorials, explanations, and conversational assistance.

Key advantages of Gemini AI:

• Context-aware understanding of problems and code

• Natural language interaction for doubt resolution

• Generation of step-by-step explanations and solutions

• Enhances personalized learning experience

5.3 Database and Tools

1. **MongoDB (MongoDB Atlas)**

   The project uses **MongoDB Atlas** as the primary database to store and manage application data efficiently. MongoDB is a cloud-based NoSQL database that is well suited for modern web applications requiring flexibility, scalability, and high performance. It securely stores essential information such as user profiles, coding problems, code submissions, AI learning sessions, chatbot interactions, and time-stamped activity records, ensuring reliable data management across the platform.

   MongoDB's document-oriented data model allows data to be stored in flexible JSON-like documents, making it ideal for handling dynamic and evolving data structures. This flexibility is particularly useful for storing AI-generated responses, learning history, and submission metadata, which may vary in format and size. Unlike traditional relational databases, MongoDB enables easy schema updates without complex migrations, supporting rapid development and future enhancements.

   MongoDB Atlas provides built-in cloud scalability, high availability, and automated backups, ensuring data reliability even as the number of users grows. Its integration with Node.js and Express.js enables fast data access and efficient API communication. The database also supports indexing and aggregation features, which help in generating performance analytics and tracking user progress effectively.

   Key Benefits of MongoDB Atlas:

   - Secure storage of user data, problems, submissions, and AI interactions
   - Flexible NoSQL document-based data structure
   - Efficient handling of dynamic and AI-generated content
   - High scalability and cloud reliability
   - Built-in security features and access control
   - Seamless integration with Node.js and cloud services

SYSTEM DESIGN

6.1 Use Case Diagram

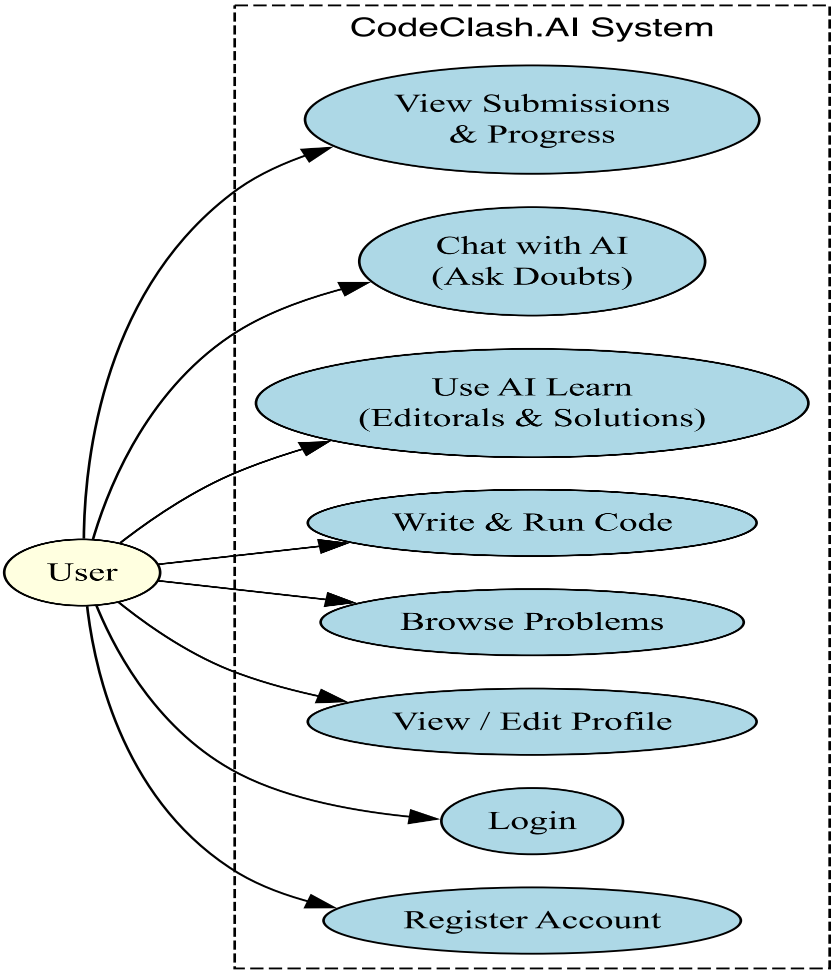
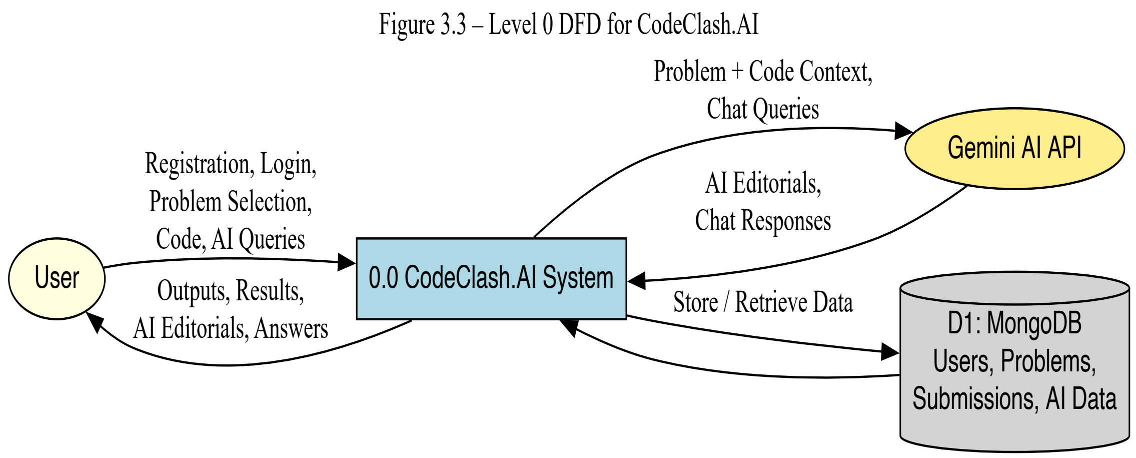


Fig. 6.1.1

The Use Case Diagram illustrates how users interact with **CodeClash.AI**, an AI-powered coding and learning platform. Since the system currently supports only end users, all interactions are performed by the user without any separate administrative role. Users can register, log in, manage their profile, browse coding problems, write and execute code, access AI Learn editorials, interact with the AI chatbot, and view their submission history and learning progress.

Several use cases are connected through <> and <> relationships. The *Login* use case includes *Validate Credentials* to ensure secure authentication. Selecting a coding problem includes choosing the topic and difficulty level. Writing and running code includes code execution and result evaluation. Accessing the AI Learn feature includes generating editorials and solution explanations using Gemini AI. The *Reset Password* use case extends the authentication process, as it is invoked only when required. This diagram highlights the user-centric design of CodeClash.AI and demonstrates how AI-assisted learning is seamlessly integrated into the coding workflow.

6.2 Data Flow Diagram (DFD)



DFD Level 0

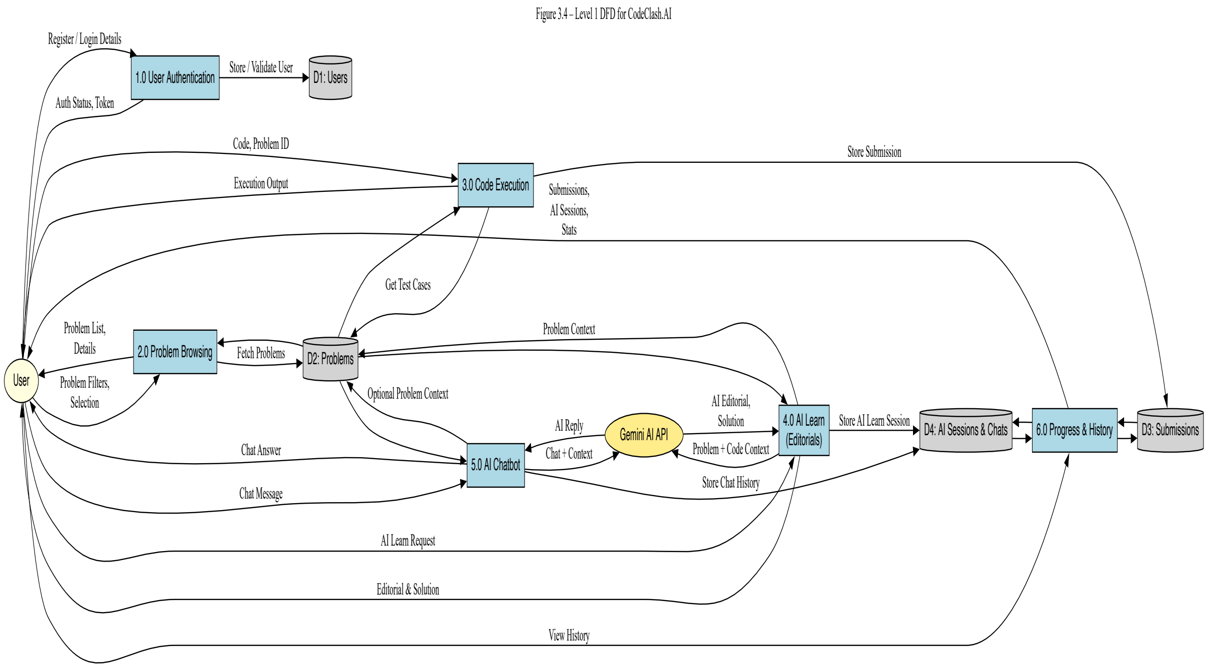
Fig. 6.2.1

DFD Level 0 represents the entire **CodeClash.AI** system as a single process. It illustrates how the user interacts with the platform and how the system exchanges data with the database and external AI services. At this level, the internal workings of the system are not shown, providing a high-level overview of data flow.

- The user sends requests such as registration, login, problem selection, code submission, and AI learning queries to the system.

- The system processes the user code, interacts with the AI module for editorials or chatbot responses, and returns execution results and AI-generated guidance to the user.

• The system stores user information, submissions, AI interactions, and learning history in the database and retrieves stored data whenever required.



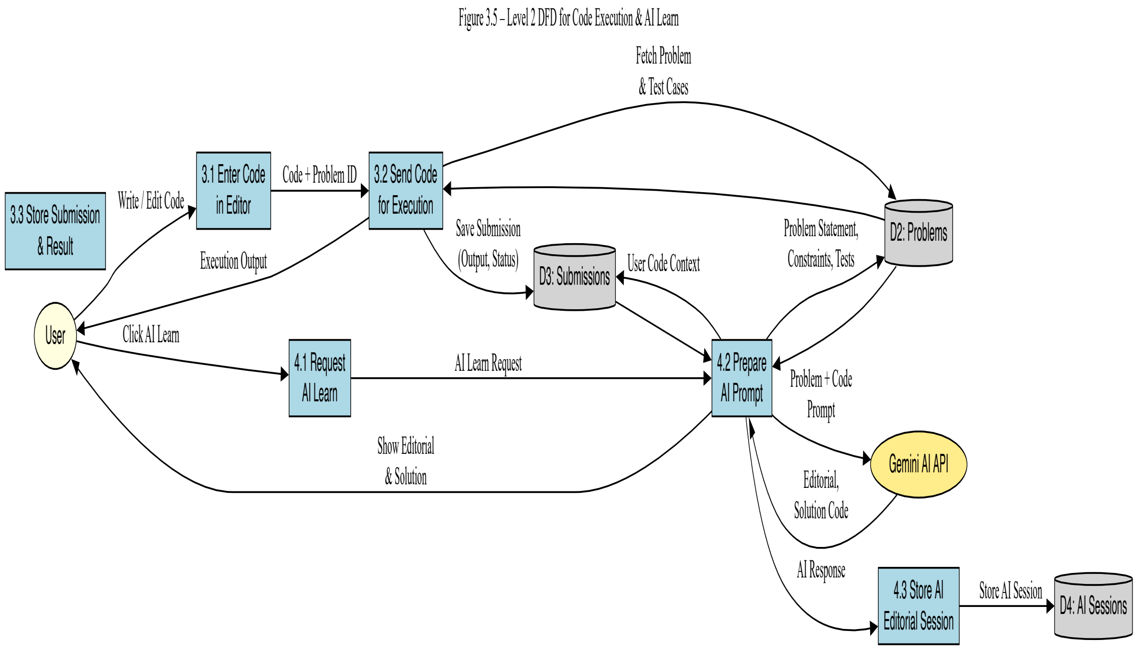
DFD Level 1

Fig. 6.2.2

DFD Level 1 breaks the system into major internal processes of **CodeClash.AI** and shows how data flows between them:

1. **User Authentication** — verifies user registration and login credentials and grants access to the platform.
2. **Problem Management** — retrieves coding problems based on selected topic and difficulty level.
3. **Code Execution** — processes user-submitted code and generates execution output or errors.
4. **AI Processing** — analyzes problem context and user code to generate AI Learn editorials and chatbot responses.

The user receives execution results, AI-generated explanations, and guidance, while the database stores user details, submissions, AI interactions, and learning history. This level explains how data flows between the main functional modules within the system.



DFD Level 2

Fig. 6.2.3

DFD Level 2 expands the **Code Execution and AI Learning** stages into detailed subprocesses:

1. **Capture User Code** — retrieves the code written by the user in the in-browser editor.
2. **Fetch Problem Context** — obtains the problem statement, constraints, and test cases from the database.
3. **Execute Code** — runs the submitted code and captures output or error information.
4. **AI Context Preparation** — combines user code and problem details to prepare an AI prompt.
5. **Generate AI Response** — uses Gemini AI to produce editorials, explanations, or chatbot replies.

The output includes execution results**, AI-generated** guidance returned to the user, and learning data stored in the database. This level shows the detailed, step-by-step data flow inside the AI-assisted coding and learning process.

6.3 ER Diagram

ER Diagram – CodeClash.AI Data Model

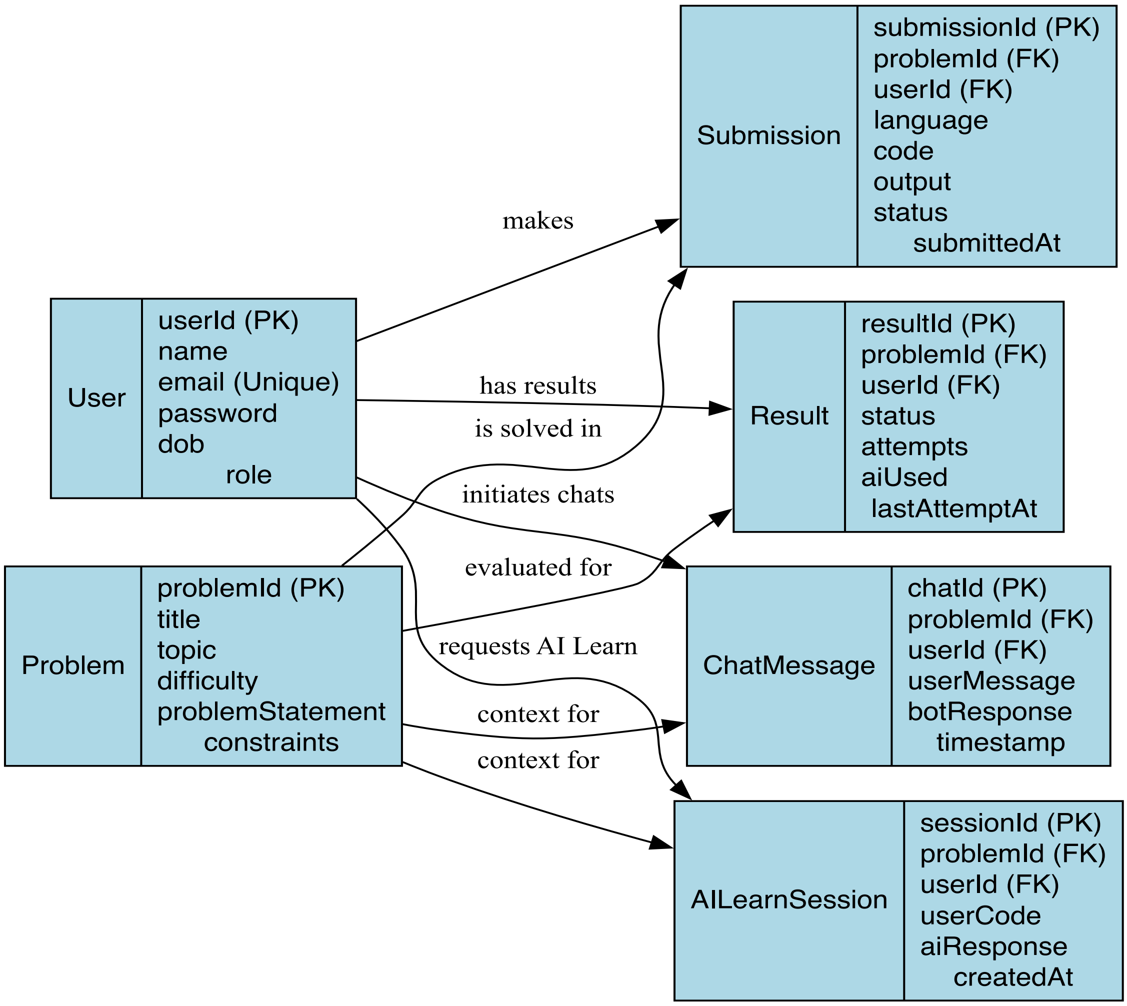


Fig. 6.3.1

The ER Diagram represents the database structure of **CodeClash.AI** and illustrates how different entities are logically related within the system.

- The **User** entity stores user information such as user ID, name, email, and authentication details.
- Each **User** can attempt multiple **Problems**, representing a one-to-many relationship.
- The **Problem** entity contains details such as problem title, topic, difficulty level, and problem statement.
- Each problem attempt results in a **Submission**, which stores the user’s code, selected programming language, execution output, and submission status.
- The system also maintains **AI Learn Sessions**, which store AI-generated editorials, explanations, and chatbot interactions linked to a specific user and problem.

This ER diagram ensures proper data organization, minimizes redundancy, and supports efficient storage and retrieval of user activity, coding progress, and AI-assisted learning data

6.4 UML Diagrams

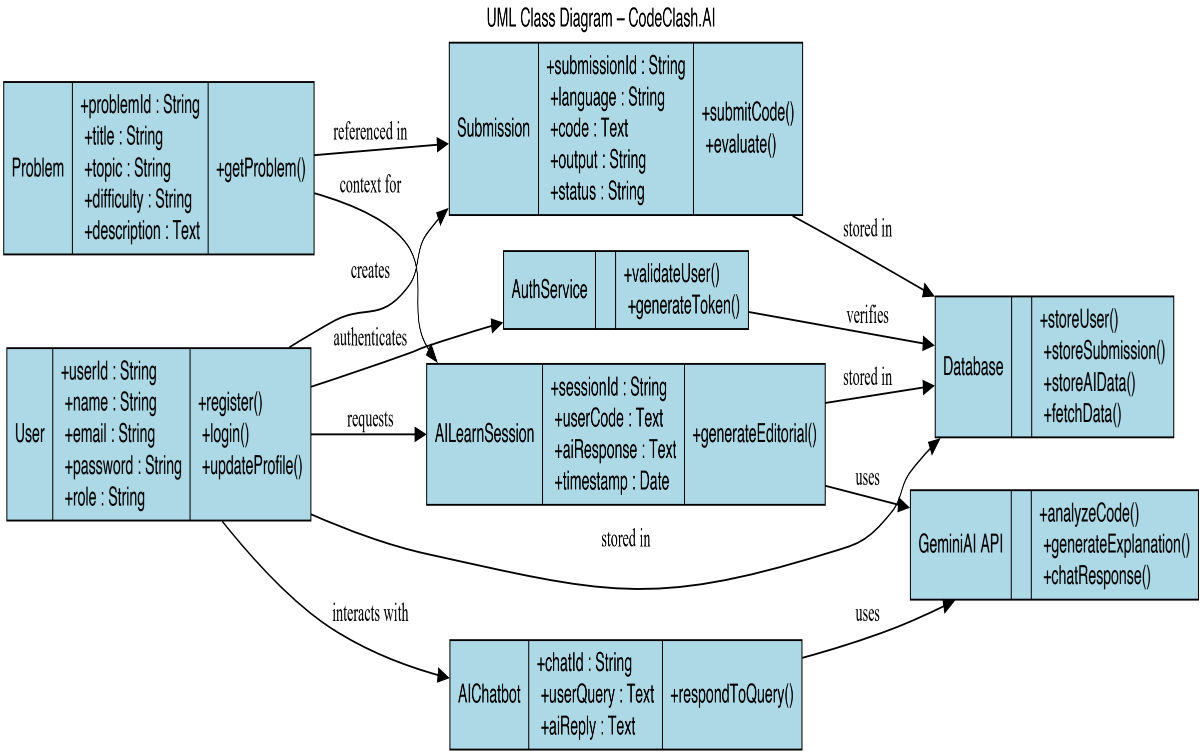


Fig. 6.4.1

The UML Class Diagram illustrates the object-oriented structure of **CodeClash.AI** and explains how different system components interact at the class level.

• The **User** class stores user-related attributes and supports operations such as registration, login, and profile management.

• A **User** interacts with the **Problem** class, which contains problem details such as title, topic, difficulty, and description.

• The **Submission** class represents user-submitted solutions and maintains information such as code, programming language, execution output, and submission status.

• The **AILearnSession** class interacts with the AI service to generate editorials, explanations, and learning guidance based on the problem context and user code.

• The **AIChatbot** class handles conversational queries from users and provides context-aware responses using AI processing.

The UML diagram clearly defines class responsibilities, relationships, and method interactions, supporting effective system implementation, scalability, and future maintenance

.

IMPLEMENTATIONS

7.1 Frontend Implementation

Technologies: ReactJS for UI development, Tailwind CSS for styling, Monaco Editor for in-browser coding, Axios for API communication, and plain JavaScript for handling editor interactions.

Key screens are implemented as reusable React components, including Login, Register, Dashboard, Problem List, Code Editor, AI Learn Panel, and AI Chatbot.

- Flow (coding and learning experience):
- The **Problem List** page fetches categorized coding problems from the backend and allows users to select problems based on topic and difficulty.
- The **Code Editor** component loads the selected problem details and provides an in-browser Monaco Editor for writing and testing code.
- User-written code is sent to the backend via **REST APIs** for execution and result generation.
- The **AI Learn** panel requests **AI-generated editorials** and step-by-step explanations from the backend and displays them alongside the editor.
- The **AI Chatbot** component enables users to ask contextual questions related to the problem or code and displays AI responses in real time.
- The **Dashboard** page displays user profile details, submission history, and learning progress using a consistent Tailwind-based layout and navigation structure.

7.2 Backend Implementation

Framework: Node.js with Express.js. The application entry point initializes middleware, API routes, database connections, and AI service integration.

- Main REST API routes:
    - POST /auth/register, /auth/login for user authentication.
    - GET /problems, /problems/:id for retrieving coding problems.
    - POST /execute for handling code execution requests.
    - POST /ai/learn for generating AI editorials and explanations.

- POST /ai/chat for handling AI chatbot queries.
- GET /user/history for fetching submission and learning history.

Server-side flow (Express API → AI Service): incoming code and problem data are validated, processed, and either executed or forwarded to the AI module. Responses from Gemini AI are parsed, formatted, and returned to the frontend. All relevant data is logged and stored for progress tracking

7.3 Database Design

- Persistent storage is implemented using MongoDB Atlas as a cloud-based NoSQL database.
- Core collections include:
- users: { userId, name, email, passwordHash, role, createdAt }.
- problems: { problemId, title, topic, difficulty, problemStatement, constraints, testCases }.
- submissions: { submissionId, userId, problemId, language, code, output, status, submittedAt }.
- ai_sessions: { sessionId, userId, problemId, userCode, aiResponse, createdAt }.
- MongoDB's flexible document structure allows efficient storage of dynamic AI-generated content and user activity data. Indexing and aggregation queries are used to support fast retrieval and progress analysis

7.4 API Integration

External services and libraries used for integration include:

- Gemini AI API for AI Learn editorials, solution explanations, and chatbot responses.
- MongoDB Atlas for secure and scalable cloud data storage.
- RESTful APIs for communication between frontend and backend.
- JWT for secure authentication and authorization.
- Axios on the frontend for handling HTTP requests and responses

.

APPLICATION DEMONSTRATIONS

8.1 User Interface Screens

8.1.1 Home Page

Fig. 8.1.1: Home Page

- Fig. 8.1.1 shows the homepage of the CodeClash.AI app. It guides users to an AI-powered coding and learning experience.

8.1.2 Login Page

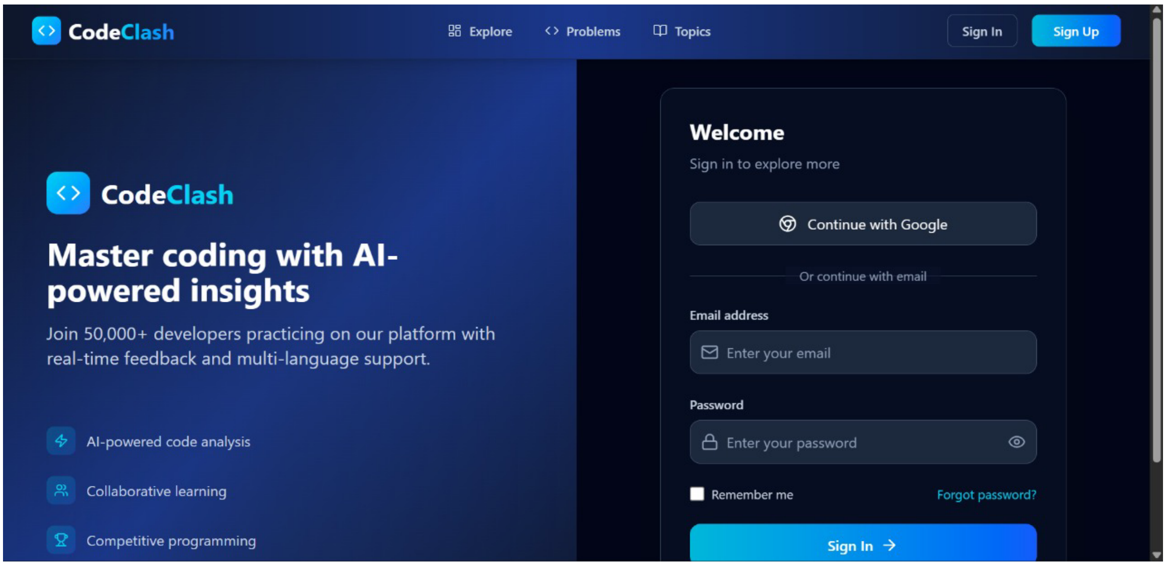


Fig. 8.1.2: Login Page

- In Fig. 8.1.2, the login page of the CodeClash.AI platform. Users enter their email and password to continue their training. Progress and stats are synced securely through MongoDB.

8.1.3 Sign Up Page

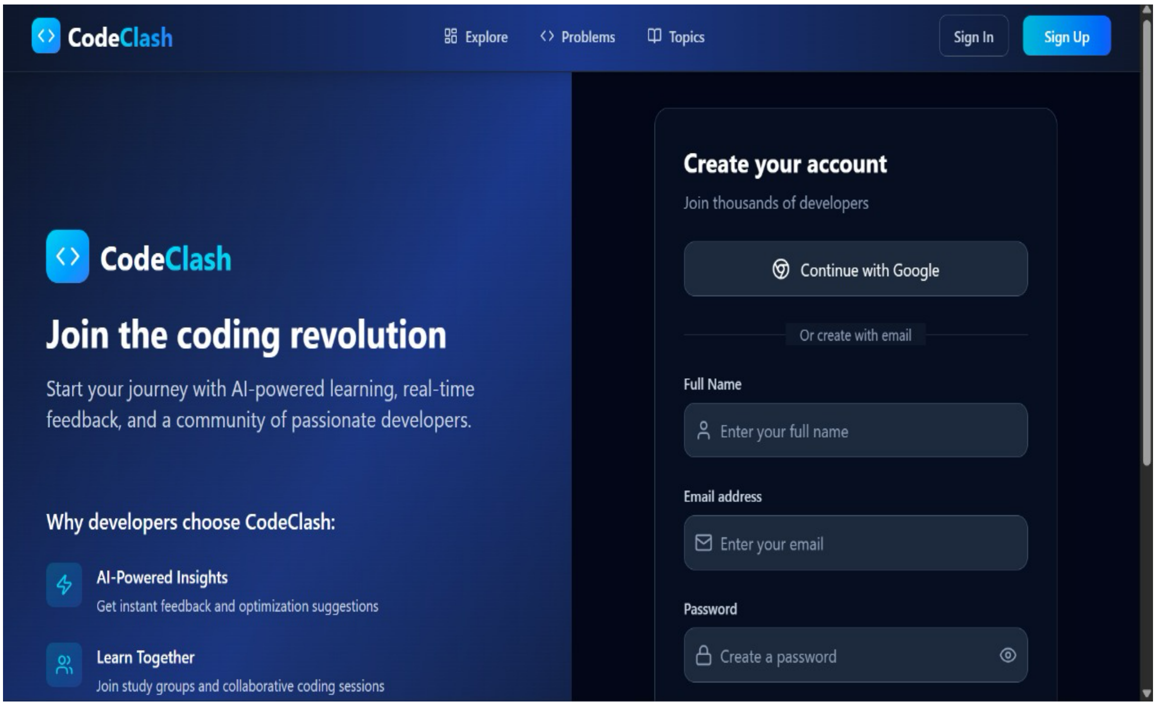


Fig. 8.1.3: Sign Up Page

- In Fig. 8.1.3 - Users can sign up with their credentials, email and a password.

8.1.4 User Profile Page

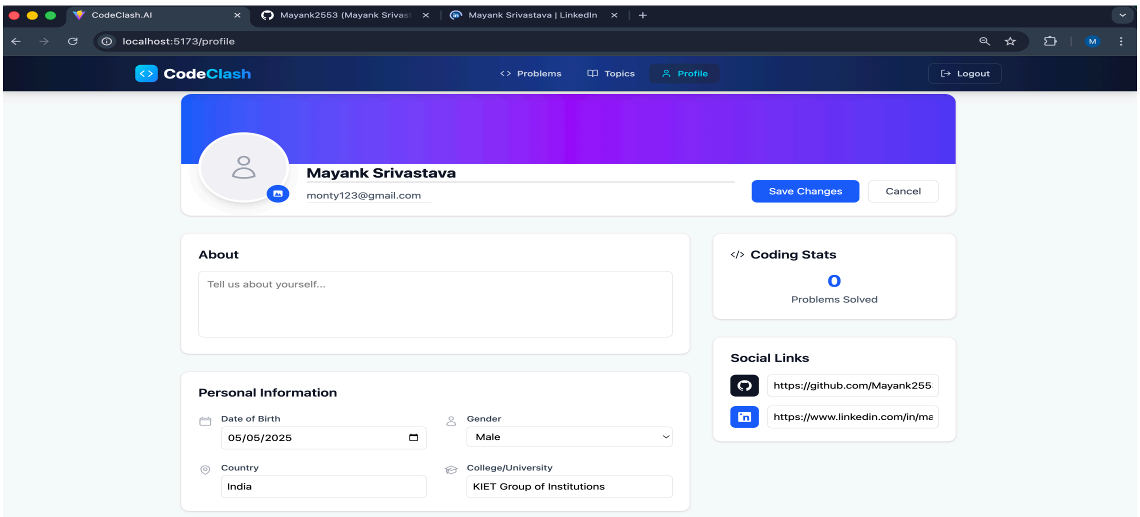


Fig 8.1.4 - User Profile Page where users can see and edit their profile information.

8.2 Problems List Interface

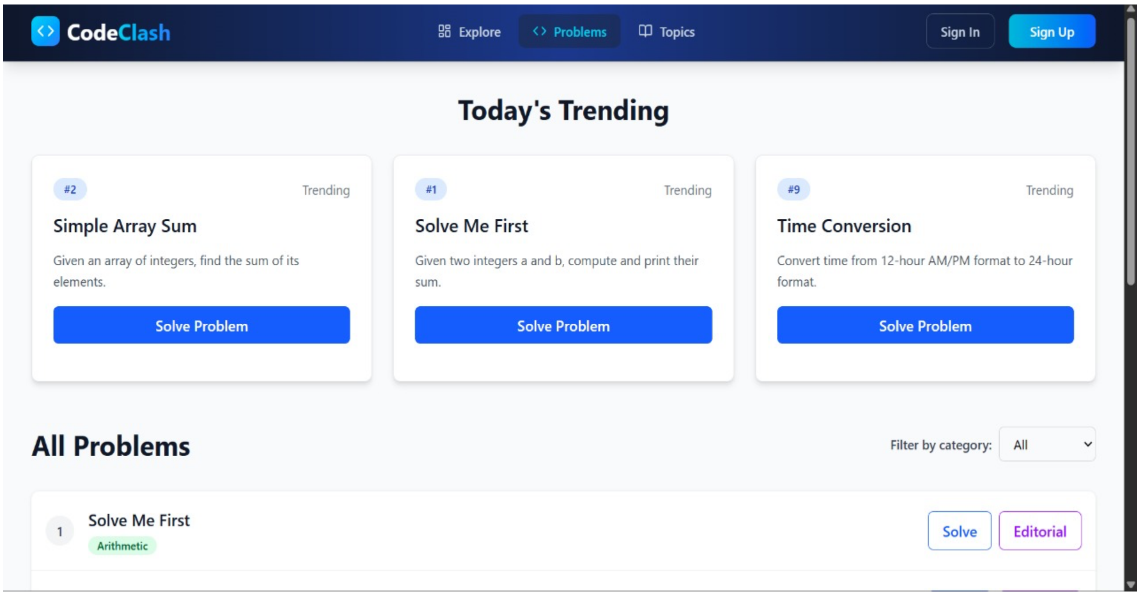


Fig 8.2.1 - Problems List Page

- Fig 8.2.1 - It displays users the list of problems they can choose from to attempt and learn, along with .

8.3 Coding Platform

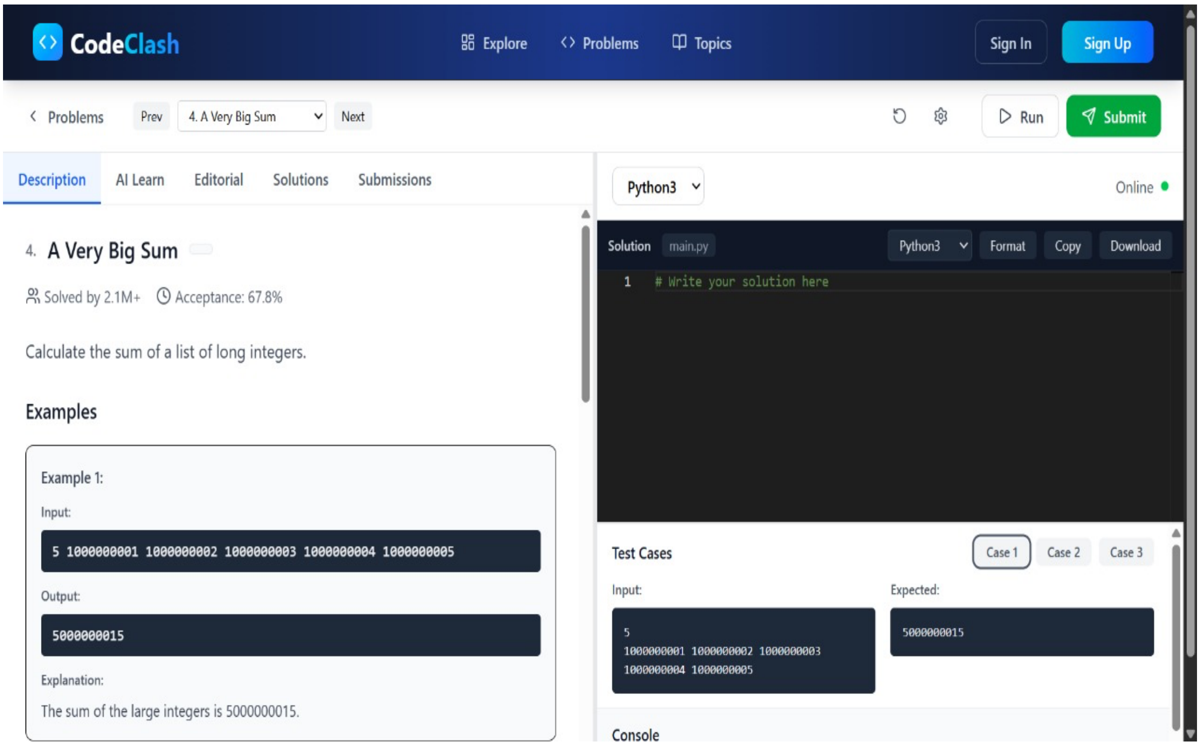


Fig. 8.3.1: Code Editor Overview

- The User can read the problem description and choose their preferred programming language to start coding the solution.

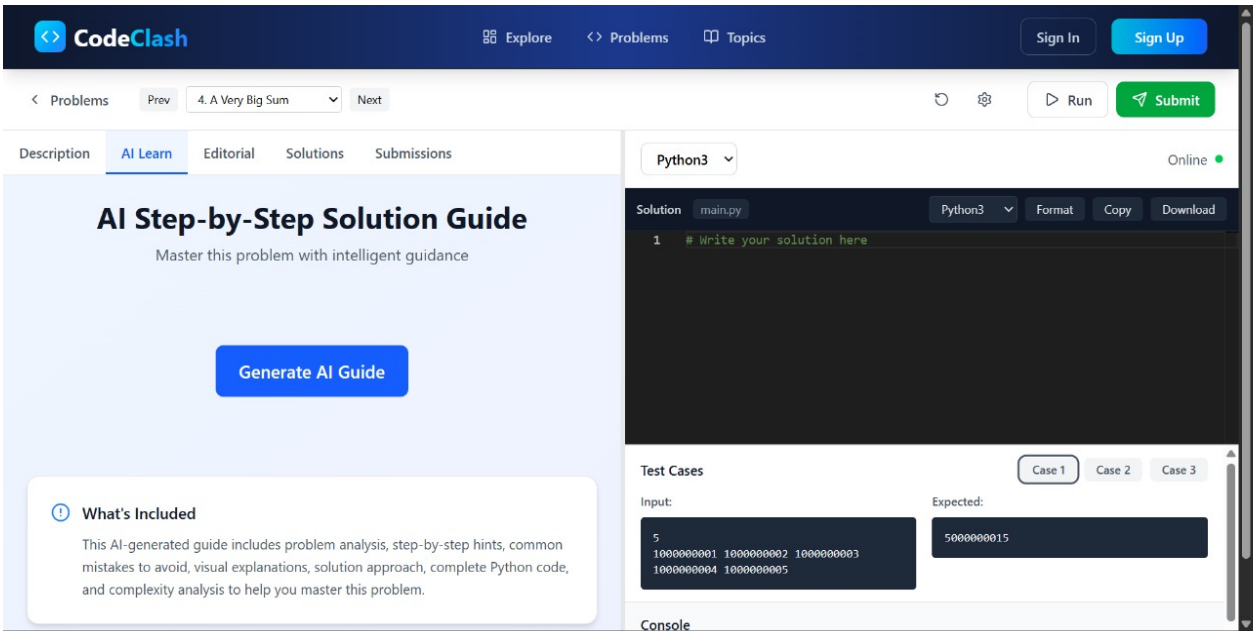


Fig. 8.3.2: AI Learn Panel

- In Fig.8.3.2, Users can generate AI guide to the problem, making use of the Gemini AI.

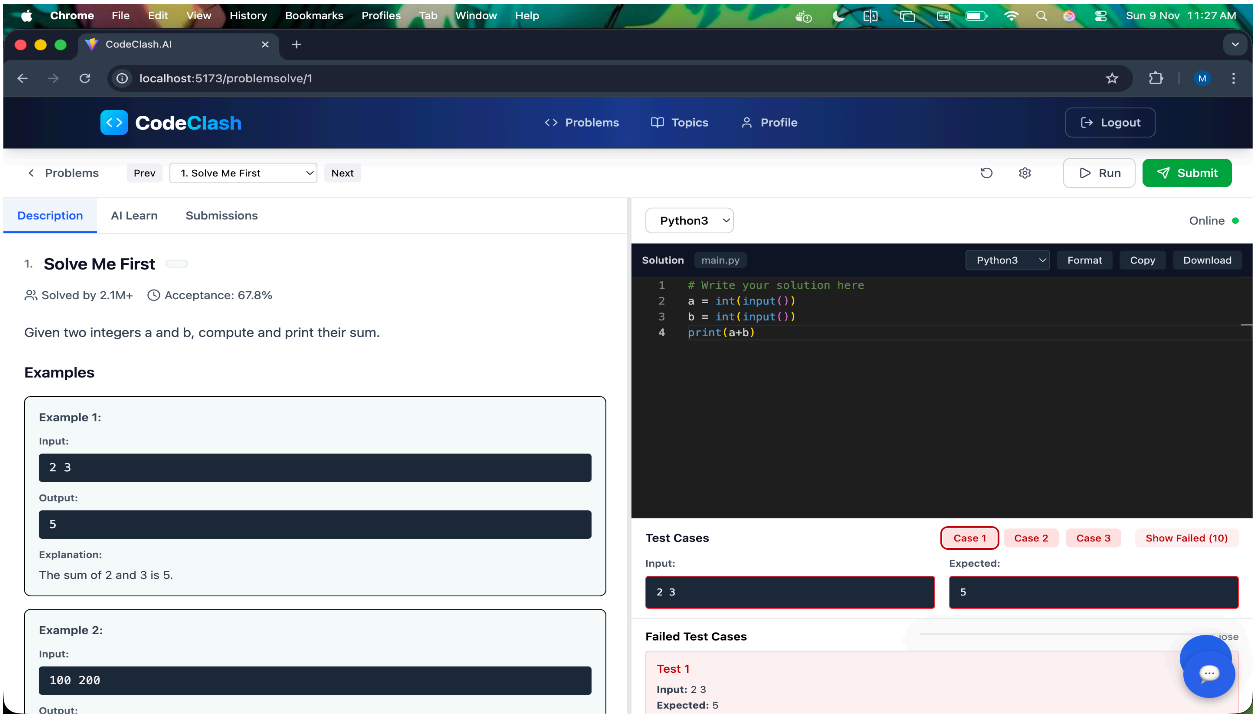


Fig. 8.3.3 : Code Execution Screen

- This screen shows the result of the code Execution made successful using Docker in the backend to execute code in different language.

8.4 AI Chatbot

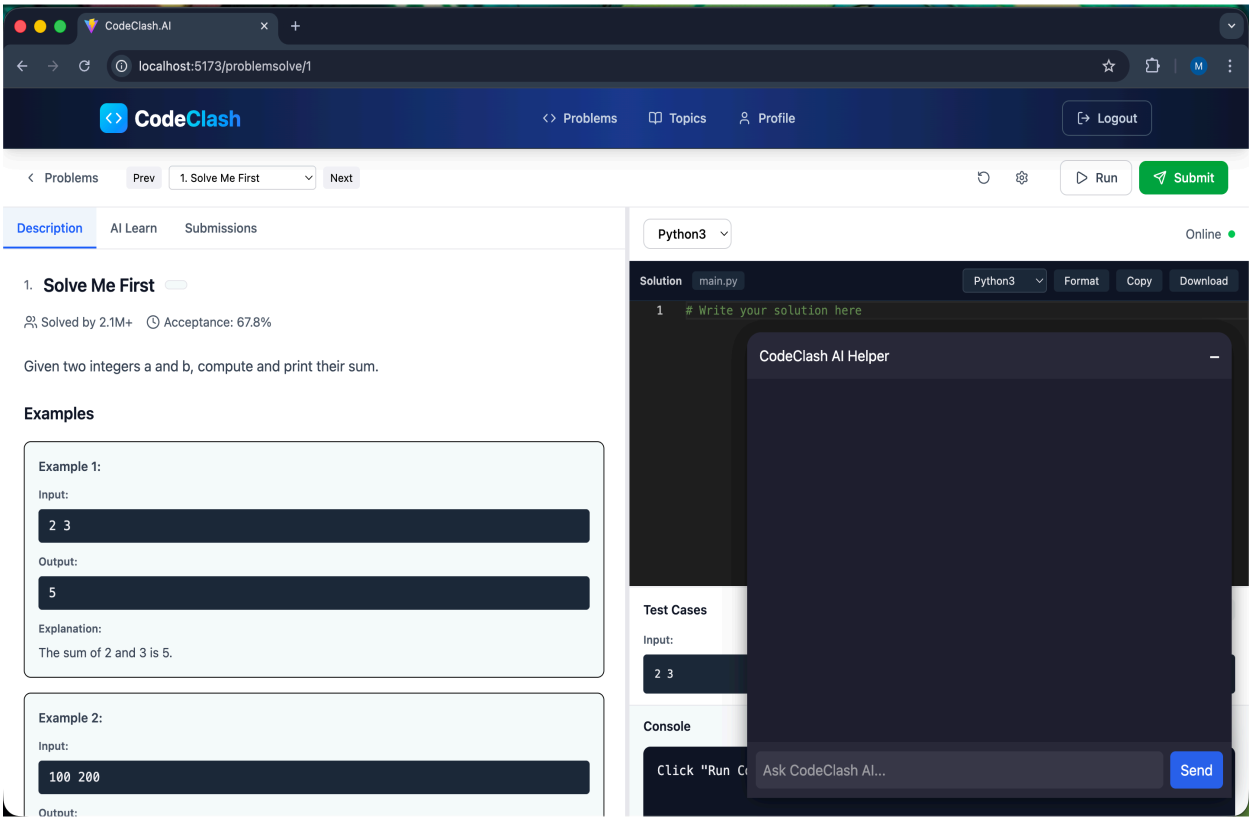


Fig. 8.4.1 : AI Chatbot

- In fig. 8.4.1, the screen shows the AI chatbot feature accessible to users through tiny toggle button on the lower right corner of the editor screen.

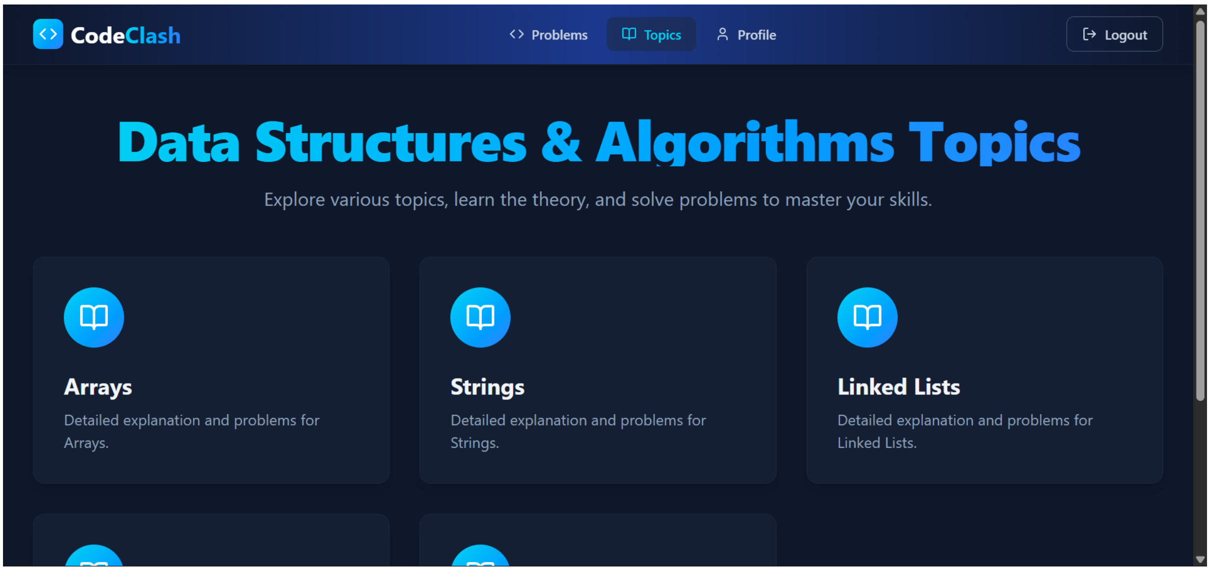


Fig. 8.4.2 AI Blog Generator (Topics to learn from beginning to advanced)

- Fig 8.4.2 - Generating AI Blogs for each topic based on level and goal

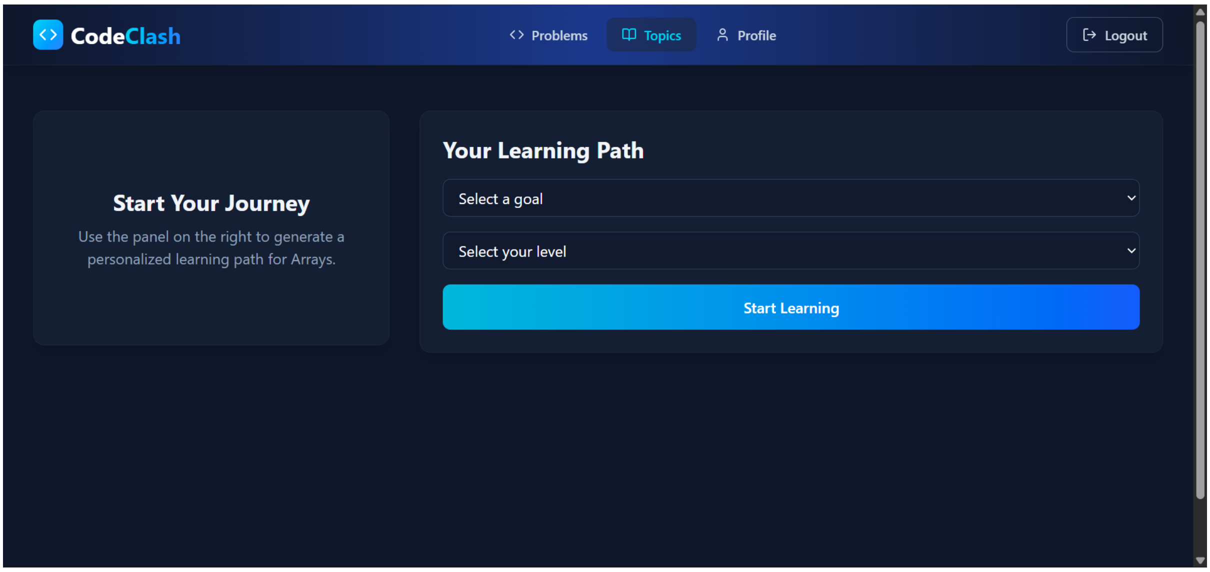


Fig. 8.4.3 Selecting the goal and level of learning

- Fig 8.4.3 - Selecting the goal and level of learning for generating AI Blog.

TESTING

9.1 Testing Strategies

Testing is a critical phase in the development of **CodeClash.AI**, ensuring system reliability, accuracy, and smooth user experience. Multiple testing strategies were applied to validate frontend components, backend services, AI integration, and overall system behavior. These strategies helped identify issues early, improve performance, and ensure a stable and effective AI-powered coding and learning platform.

A. Unit Testing: Unit testing focused on validating individual functional components of the system.

Key Points

- Tested user authentication and authorization modules.
- Verified problem retrieval and submission handling logic.
- Checked code execution functionality and output generation.
- Validated AI Learn and AI Chatbot request handling.
- Identified and fixed component-level issues before integration.

B. Integration Testing: Integration testing verified interaction between different system modules.

Key Points

- Checked communication between frontend and backend services.
- Verified data flow between code editor, execution module, and AI services.
- Ensured AI responses were correctly processed and displayed on the frontend.
- Validated API request and response formats.

C. System Testing: System testing evaluated the platform as a complete integrated solution.

Key Points

- Tested complete user flow from registration to problem solving.
- Validated AI Learn and chatbot features during real usage scenarios.
- Ensured all functionalities worked as expected under normal user behavior.
- Checked UI responsiveness and navigation flow.

D. Performance & Load Testing: Performance testing assessed system stability under varying workloads.

Key Points

- Measured response time for code execution and AI-generated results.
- Simulated multiple user requests to evaluate backend performance.
- Ensured AI integration remained responsive without delays.
- Evaluated system behavior under continuous usage.

E. Security Testing: Security testing ensured protection of user data and system integrity.

Key Points

- Tested JWT-based authentication and token validation.
- Verified access control for protected API endpoints.
- Ensured secure communication between frontend and backend.
- Protected user data from unauthorized access.

F. Usability Testing - Usability testing focused on providing a smooth and user-friendly learning experience.

Key Points

- Evaluated clarity and usability of the user interface.
- Tested ease of navigation across different sections of the platform.
- Collected feedback from test users to improve usability.
- Ensured AI responses and editorials were easy to understand and helpful.

9.2 Test Cases and Results

This section presents the major test cases executed during the evaluation of the **AI-Powered Self-Defense Training Platform**. Each test case was designed to verify the correct functioning of system modules such as user authentication, AI pose detection, real-time feedback, and database connectivity. The results show that the system performs reliably under various scenarios and meets the functional requirements of the project.

A. Functional Test Cases

1. User Registration & Login Test

Test Case ID	**TC-01**
Objective	To verify if users can register and log in securely.
Input	Email, password, user details
Expected Output	Successful account creation and login
Actual Output	Worked as expected
Status	Passed

Table 9.2.1: User Registration & Login Test

2. Problem Retreival Test

Test Case ID	**TC-02**
Objective	To verify if the coding problems are fetched and displayed correctly.
Input	Problem request by topic and difficulty
Expected Output	Correct List of Problems displayed
Actual Output	Problems displayed successfully
Status	Passed

Table 9.2.2 - Problem Retreival Test

3. Code Execution Test

Test Case ID	TC-03
Objective	To ensure user submitted code executes correctly.
Input	Code submissions with test cases
Expected Output	Correct output or error message
Actual Output	Output generated successfully
Status	Passed

Table 9.2.3: Code Execution Test

B. AI Processing Test Cases

4. AI Learn Editorial GenerationTest

Test Case ID	TC-04
Objective	To verify AI-generated editorials and explanations.
Input	Problem Context and User Code
Expected Output	Step-by-step explanation and solution
Actual Output	Editorial Generated Successfully
Status	Passed

Table 9.2.4: AI Learn Editorial Generation Test

5. AI Chatbot Interaction Test

Test Case ID	**TC-05**
Objective	To verify AI chatbot responses for user queries.
Input	User question related to code or problem.
Expected Output	Relevant AI generated response
Actual Output	Accurate and contextual reply
Status	Passed

Table 9.2.5: AI Chatbot Interaction Test

C. Database Test Cases

6. MongoDB Data Storage Test

Test Case ID	**TC-06**
Objective	To verify storage of submissions and AI Interactions.
Input	Completed code submissions and AI usage.
Expected Output	Data stored successfully in MongoDB
Actual Output	All records stored successfully
Status	Passed

Table 9.2.6: MongoDB Data Storage Test

D. Performance Test Cases

7. Concurrent-User Load Test

Test Case ID	**TC-07**
Objective	To check system performance with multiple users simultaneously.

Test Case ID	**TC-07**
Input	Simulatenous User Requests
Expected Output	Smooth processing without lag
Actual Output	System remained stable
Status	Passed

Table 9.2.7: Concurrent-User Load Test

E. Usability Test Cases

8. User Interface Navigation Test

Test Case ID	**TC-08**
Objective	To verify that users can easily navigate through features.
Input	Buttons, menus, training screens
Expected Output	Clear, smooth navigation
Actual Output	All UI components worked properly
Status	Passed

Table 9.2.8: User Interface Navigation Test

RESULTS AND ANALYSIS

10.1 Performance Evaluation

The **CodeClash.AI** platform was evaluated across multiple performance parameters to assess efficiency, responsiveness, and system reliability under different usage scenarios. The evaluation focused on code execution speed, AI response time, system responsiveness, data handling efficiency, and scalability. The results indicate that the platform performs efficiently and meets the technical requirements necessary for a smooth and effective AI-powered coding and learning experience.

A. Code-Execution Performance

Efficient code execution is critical for providing immediate feedback to learners during practice.

Key Observations

• Average code execution response time remained within acceptable limits for standard problem sizes.

• Execution results were generated quickly without noticeable delay.

• No execution failures were observed during repeated submissions.

The code execution workflow handled multiple test cases efficiently, allowing users to iterate and refine solutions smoothly

B. AI Response Accuracy and Reliability

The AI components were evaluated for correctness and relevance of editorials and chatbot responses.

Key Observations

- AI Learn editorials accurately explained problem logic and solution approaches.
- Chatbot responses were context-aware and relevant to user queries.
- Minor variations in response depth were observed based on query complexity.

The AI system consistently provided meaningful guidance, enhancing conceptual understanding.

C. System Scalability & Load Performance

The platform was tested with increasing user loads to assess its stability and scalability.

Key Observations

- Handled multiple concurrent sessions without performance degradation.
- MongoDB database responded quickly under load conditions.
- No crashes observed during extended session testing (20+ minutes).

Scalability tests confirmed that the system can support many users with minimal resource consumption.

D. User Interface & Responsiveness

The UI was tested for responsiveness across different screen sizes and devices.

Key Observations

- Smooth navigation on laptops and mobile devices.
- Buttons, menus, and feedback visuals loaded without delay.
- Frontend remained responsive during AI processing.

10.2 Result Analysis

The **CodeClash.AI** platform was evaluated across multiple parameters to analyze its functional accuracy, system performance, user experience, and AI reliability. The analysis of results indicates that the platform successfully meets its project objectives by delivering an effective AI-assisted coding and learning experience with accurate code execution, meaningful AI guidance, and smooth user interaction.

A. Functional Results

The system performed all major functions—user registration and login, problem selection, code execution, AI Learn access, chatbot interaction, and progress tracking—smoothly and consistently. Each module responded correctly to user actions, with no major errors or system crashes observed during the testing phase.

Findings

- Seamless user authentication and secure access to protected features
- Reliable problem retrieval and code execution workflow
- AI Learn editorials and chatbot responses generated without failure

B. AI Performance Analysis

The AI components demonstrated strong performance in analyzing problem context and user-submitted code. Gemini AI consistently produced relevant explanations, solution approaches, and accurate responses to user queries, enhancing conceptual understanding.

Findings

- AI-generated editorials were clear, structured, and context-aware

- Chatbot responses accurately addressed user doubts related to code and logic
- AI response time remained within acceptable limits for real-time interaction

C. System Stability and Load Analysis

The platform was tested under different load conditions to assess scalability. Even when multiple users accessed the system simultaneously, there were no slowdowns or performance drops.

Findings

- Real-time processing remained stable during load testing
- No frame drops or lag during continuous 15–20-minute sessions
- MongoDB maintained fast read/write operations

D. User Experience (UX) Analysis

User testing sessions confirmed that the platform is intuitive and easy to navigate. The visual feedback system improved user engagement and helped learners correct their movements effectively.

Findings

- High user satisfaction due to clean UI and smooth navigations.
- Easy interaction with the Code Editor, AI Learn panel and chatbot.
- AI explanations and feedback were easy to understand and helpful.

CONCLUSION

The **CodeClash.AI** platform successfully achieves its objective of providing an intelligent, interactive, and accessible solution for learning and practicing coding concepts. By integrating **Artificial Intelligence (Gemini AI)** directly into the coding environment, the system enables users to write code, understand problem logic, and receive instant explanations and guidance. This approach transforms traditional coding practice into a more engaging and concept-driven learning experience.

The project effectively addresses common challenges faced by learners on conventional coding platforms, such as lack of personalized guidance, fragmented learning resources, and limited conceptual clarity. With AI-powered editorials and a context-aware chatbot, users can resolve doubts instantly without relying on external tutorials. The web-based nature of the platform allows learners to practice anytime and anywhere using standard devices, making coding education more flexible and accessible to users from diverse academic backgrounds.

The platform demonstrates strong technical performance through the successful implementation of modern web technologies such as **ReactJS, Node.js, Express.js, MongoDB**, and **Gemini AI**. Testing results confirm that the system performs reliably, offering smooth code execution, accurate AI-generated explanations, responsive user interaction, and secure data management. The modular system architecture ensures efficient communication between frontend, backend, database, and AI services.

Overall, this project highlights the practical application of artificial intelligence in the field of programming education. **CodeClash.AI** not only enhances technical learning and problem-solving skills but also promotes self-paced, guided learning. The platform provides a strong foundation for future enhancements in AI-assisted education systems and intelligent learning platforms

.

FUTURE SCOPE

The **CodeClash.AI** platform has significant potential for future enhancement and expansion as artificial intelligence and educational technologies continue to evolve. While the current system focuses on AI-assisted coding practice, editorials, and real-time doubt resolution, several advanced features can be introduced to further enrich the learning experience. In the future, more advanced AI models can be integrated to provide deeper code analysis, optimized algorithm recommendations, and personalized learning paths based on individual user performance and learning patterns.

One major area of expansion is the introduction of **adaptive learning systems**, where the difficulty level of problems automatically adjusts according to the user's strengths and weaknesses. This would enable a more personalized and efficient learning journey. Additionally, support for **multiple programming languages** such as Python, C++, and Java can be added to broaden the platform's usability and appeal to a wider audience.

The platform can also be extended into a **mobile application** for Android and iOS devices, allowing learners to practice coding and access AI guidance on the go. Integration with **Learning Management Systems (LMS)** and college portals can further enhance its adoption in academic environments. Multi-language support can be implemented to make the platform accessible to users from different regions and linguistic backgrounds.

Furthermore, future versions of CodeClash.AI may include **advanced analytics dashboards**, instructor or mentor roles, collaborative learning features, and AI-driven interview preparation modules. These enhancements would increase user engagement and transform CodeClash.AI into a comprehensive ecosystem for intelligent, AI-powered programming education

.

REFERENCES

[1] M. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, **Node.js in Action**, Manning Publications, 2017.

[2] Express.js Documentation Team, **“Express – Fast, Unopinionated, Minimalist Web Framework for Node.js,”** OpenJS Foundation, 2023.

[3] React Core Team, **“React: A JavaScript Library for Building User Interfaces,”** Meta Platforms Inc., 2023.

[4] MongoDB Inc., **“MongoDB Manual: NoSQL Database Documentation,”** MongoDB Documentation, 2023.

[5] Google AI, **“Gemini API Documentation: Generative AI for Developers,”** Google Developers, 2024.

[6] Microsoft Corporation, **“Monaco Editor: Web-Based Code Editor,”** Microsoft Documentation, 2023.

[7] Auth0 Documentation Team, **“JSON Web Tokens (JWT) for Secure Authentication,”** Auth0 Docs, 2023.

[8] R. Richardson, **RESTful Web APIs**, O’Reilly Media, 2013.

[9] A. Grigorik, **High Performance Browser Networking**, O’Reilly Media, 2013.

[10] World Wide Web Consortium (W3C), **“HTML5 Specification,”** W3C Recommendation, 2014.

[11] ISO/IEC, **“Software Engineering — Software Life Cycle Processes,”** ISO/IEC 12207, 2017.

[12] I. Sommerville, **Software Engineering**, 10th ed., Pearson Education, 2016.

[13] Mozilla Developer Network (MDN), **“JavaScript Guide,”** Mozilla Foundation, 2023.

[14] Tailwind Labs, **“Tailwind CSS Documentation,”** Tailwind CSS Docs, 2023.

[15] Postman Documentation Team, **“Postman API Testing and Development,”** Postman Learning Center, 2023.